Trabajo Práctico Obligatorio - Programación Orientada a Objetos

Etapa 1: Modelado y Diseño del sistema

Índice

Propuesta e Integrantes 1
Presentación 1
Alcance funcional 2
Entidades del sistema 2
Actores del sistema 3
Requerimientos funcionales 4
Reglas de Negocio 5
Supuestos 7
Diagrama de Casos de Uso 9
Casos de usos 9
Diagrama de Clases Inicial 15
Diagrama de Secuencia 17
Aplicación de principios SOLID 17
Aplicación de principios GRASP 18
Diagrama de Clases Final 21
Nota de cambios respecto al diagrama de clases inicial 21

Propuesta e Integrantes

La propuesta seleccionada fue la opción C - FutbolStats (gestion y analitica de torneos de futbol).

Integrantes:

- Fausto Fernandez
- Ignacio Mendes
- Luka Kishima

Presentación

FutbolStats es una plataforma para administrar torneos de fútbol profesional, registrar partidos en vivo, conservar estadísticas, notificar hechos relevantes y producir reportes y proyecciones. Este documento desarrolla la Etapa 1: análisis del problema, alcance funcional, entidades, actores, requerimientos funcionales, reglas

de negocio, supuestos, diagrama de casos de uso, casos de uso, diagrama de clases inicial, diagrama de secuencia, aplicación de SOLID y GRASP y diagrama de clases final.

Alcance funcional

- Administrar torneos, clubes, estadios, jugadores, entrenadores y árbitros.
- Inscribir clubes con plantel ya armado y generar el fixture del torneo.
- Programar partidos con local/visitante, fechayhora, estadio y árbitros.
- Registrar eventos durante el juego y cerrar el estado del partido.
- Notificar goles, tarjetas rojas y finales de partido.
- Gestionar contratos de jugadores y patrocinios.
- Consultar tabla de posiciones, goleadores y valla menos vencida.
- Calcular xG y probabilidad de clasificación mediante estrategias intercambiables.

Entidades del sistema

Entidad	Descripción
Torneo	Competencia con sus clubes inscriptos, su estado y fixture.
Fixture	Organiza los partidos de un torneo.
Club	Equipo que participa de los torneos.
Estadio	Lugar donde se juegan los partidos.
Persona	Clase base de Jugador, Entrenador y Árbitro
Jugador / Entrenador / Árbitro	Personas que forman parte del sistema.
Inscripción	Registro de un club en un torneo, con su plantel.
Contrato	Vínculo entre un jugador y un club, con las fechas.
Partido	Encuentro entre dos clubes dentro de un torneo.
Nomina	Titulares y suplentes de un club para un partido.

Terna arbitral	Grupo de árbitros elegidos para un partido.
Evento	Clase abstracta para Gol, Remate, Tarjeta, Cambio, Falta y Anulación.
Sanción	Suspensión de un jugador por tarjetas.
Sponsor / Patrocinio	Empresa que patrocina y el acuerdo con un club o torneo.
Suscriptor	Persona que sigue clubes y elige por qué canales recibe avisos.
Notificacion / Envío	Aviso de un hecho del partido y el registro de cada envío.
Canal	Medio por el cual se envía el aviso (app móvil, correo o SMS).

Actores del sistema

Actor	Qué hace en el sistema
Administrador	Carga y mantiene torneos, clubes, estadios, jugadores, entrenadores y árbitros. Inscribe clubes, genera el fixture, programa partidos y gestiona contratos y patrocinios.
Operador del partido	Carga las nóminas, inicia el partido, registra los eventos en vivo, anula los eventos erróneos y cierra el partido.
Analista	Consulta la tabla de posiciones, los goleadores y la valla menos vencida, y pide proyecciones de xG y de probabilidad de clasificación.
Suscriptor	Se registra, elige qué clubes sigue y por qué canales, y recibe las notificaciones.
Canales de notificación	Entregan cada aviso al suscriptor. Pueden fallar sin frenar la carga del partido.

Requerimientos funcionales

- **RF01** - Administrador: Administrar torneos, clubes, estadios, jugadores, entrenadores y árbitros (alta, modificación, consulta y baja), y avanzar el estado de cada torneo. - CU01
- **RF02** - Administrador: Gestionar los contratos de los jugadores (alta, transferencia y rescisión), conservando el historial de clubes. - CU02
- **RF03** - Administrador: Inscribir un club en un torneo con su plantel de 18 a 30 jugadores. - CU03
- **RF04** - Administrador: Generar el fixture del torneo, todos contra todos, a partir de los clubes inscriptos. - CU04
- **RF05** - Administrador: Programar un partido con fecha y hora, estadio y terna arbitral, validando que no haya superposiciones. - CU05 y CU14
- **RF06** - Operador de partido: Cargar la nómina de cada club para un partido, con titulares y suplentes y sin jugadores suspendidos. - CU06
- **RF07** - Operador de partido: Iniciar un partido, registrar sus eventos en vivo (goles, remates, tarjetas, cambios y faltas) y cerrarlo. - CU07
- **RF08** - Operador de partido: Anular un evento cargado por error, sin borrarlo. - CU08
- **RF09** - Suscriptor: Recibir notificaciones de goles, tarjetas rojas y finales de partido por los canales que eligió. - CU09
- **RF10** - Suscriptor: Registrarse y elegir los clubes que sigue y los canales por los que recibe avisos. - CU10
- **RF11** - Administrador: Registrar sponsors y sus patrocinios a clubes y torneos. - CU11
- **RF12** - Analista estadístico: Emitir la tabla de posiciones, la lista de goleadores y la valla menos vencida de un torneo. - CU12
- **RF13** - Analista estadístico: Calcular proyecciones de goles esperados (xG) y de probabilidad de clasificación, con el método que elija. - CU13

Reglas de Negocio

- **RN01** - Estados del torneo - PLANIFICADO, EN CURSO y FINALIZADO. Solo se permite avanzar al estado siguiente.
- **RN02** - Inicio del torneo - Un torneo requiere al menos 4 clubes inscriptos para que comience
- **RN03** - Inscripción - Un club no puede inscribirse más de una vez en el mismo torneo, y no se permiten nuevas inscripciones cuando el torneo está EN CURSO.
- **RN04** - Fechas - La fecha y hora de cada partido deben quedar dentro del período de su torneo.
- **RN05** - Plantel - Cada plantel de una inscripción contiene entre 18 y 30 jugadores
- **RN06** - Pertenencia - Un jugador solo puede pertenecer a un club por vez dentro del mismo torneo.
- **RN07** - Historial del jugador - Los datos y estadísticas de un jugador se conservan aunque rescinda su contrato o quede libre. Cada paso por un club queda registrado con su fecha de inicio y fin.
- **RN08** - Disponibilidad del estadio - Un estadio no tiene que admitir partidos con menos de 3 horas de diferencia.
- **RN09** - Programación - Un club no puede jugar dos partidos con menos de 48 horas de diferencia.
- **RN10** - Árbitros - Un partido requiere un árbitro principal y dos asistentes.
- **RN11** - Estado del partido - PROGRAMADO, EN JUEGO, FINALIZADO, SUSPENDIDO y SIN_PROGRAMAR. Los partidos se crean sin programar y pasan a Programado cuando se les asignan los datos necesarios.
- **RN12** - Registro de eventos - Solo se pueden registrar eventos con el partido EN JUEGO. El jugador involucrado debe estar en la nómina de su club para ese partido.
- **RN13** - Tiempo de evento - El tiempo de partido es entre 0 y 120 minutos (incluido el tiempo adicional).
- **RN14** - Gol y marcador - Un gol puede ser a favor, en contra o de penal. El gol en contra suma al marcador del rival mientras que el gol de penal cuenta para el goleador si es a favor.

- **RN15** - Tarjetas - Dos amarillas en el mismo jugador en un partido equivalen a una roja. Un expulsado no puede ser sustituido y el club queda con un jugador menos.
- **RN16** - Cambios - Cada club puede realizar como máximo 5 sustituciones por partido. Un jugador sustituido no puede volver a entrar
- **RN17** - Anulación de eventos - Un evento registrado no se elimina. Si hubo un error, se corrige con un evento de anulación asociado al original.
- **RN18** - Sanciones - Un jugador que acumula 5 tarjetas amarillas en el torneo queda suspendido para la fecha siguiente, y una roja directa implica 1 fecha de suspensión. Un jugador suspendido no puede incluirse en la nómina.
- **RN19** - Impedimentos del arbitro - Un árbitro no puede dirigir a un club que es hincha declarado ni tener otro partido el mismo día.
- **RN20** - Notificaciones - Cada gol, tarjeta roja y final del partido se notifica a los suscriptores que siguen a alguno de los clubes, por los canales que eligieron, indicando partido, minuto y hecho.
- **RN21** - Fallo de canales - Si un canal falla, se registra el error y se sigue con los demás, sin frenar la carga del partido. Un canal nuevo debe poder agregarse sin modificar la lógica de los partidos.
- **RN22** - Patrocinios - Un sponsor puede patrocinar varios clubes y torneos a la vez. Cada patrocinio tiene monto, fecha de inicio y fin, y solo puede haber uno vigente por sponsor y destinatario.
- **RN23** - Sponsor principal - Un club no puede tener más de un sponsor principal vigente a la vez.
- **RN24** - Tabla de posiciones - La victoria suma 3 puntos, el empate 1 y la derrota 0, solo cuentan los partidos finalizados.
- **RN25** - Criterio de desempate en tabla de posiciones - En caso de empate en puntos, la tabla de posiciones se ordenará aplicando primero la diferencia de gol, luego los goles a favor y finalmente el resultado directo entre los equipos involucrados.
- **RN26** - Goleador y Valla menos vencida - El goleador es el jugador con más goles a favor: los de penal cuentan y los en contra no. La valla menos vencida es el club con el menor promedio de goles recibidos por partido.

- **RN27** - Goles esperados (xG) - Cada remate vale según la zona desde la que se patea: área chica 0.40, área grande 0.15, fuera del área 0.04 y penal 0.76. El xG de un jugador o de un club en un partido es la suma de sus remates.
- **RN28** - Probabilidad de clasificación - recen al menos dos métodos intercambiables: comparar los puntos proyectados con el corte de años anteriores, o simular los partidos restantes con el xG promedio de cada club. Se calcula al pedirla, no se guarda, y un método nuevo se agrega sin modificar los reportes que lo usan.

Supuestos

- **S01** - Actores - El enunciado no dice quien carga los datos, organiza el torneo ni programa los partidos: el actor “Administrador” será encargado de eso. Los eventos del partido los registra el “Operador del Partido”.
- **S02** - Datos del torneo - Cada torneo indica si es nacional o internacional, cuántos clubes clasifican y el puntaje de corte de años anteriores. Los carga el “Administrador” al crear el torneo.
- **S03** - Fixture - Se genera todos contra todos a una sola rueda. Si la cantidad de clubes es impar, en cada fecha un club queda libre. El fixture define los cruces y la localía; el estadio, la hora y la terna se asignan al programar cada partido.
- **S04** - Nómina - Antes de iniciar el partido, cada club presenta 11 titulares y hasta 7 suplentes, elegidos de su plantel inscripto en el torneo.
- **S05** - Sanciones - “Fecha siguiente” se interpreta como el próximo partido del club en ese torneo. Una expulsión por doble amarilla se trata como roja: 1 fecha de suspensión, y se notifica como tarjeta roja.
- **S06** - Desempate - “Resultado entre los clubes empatados” son los puntos que sacaron en los partidos entre ellos. Si siguen empatados o no se enfrentaron, se ordenan alfabéticamente.
- **S07** - Proyección por promedio - Los puntos proyectados son el promedio de puntos por partido multiplicado por la cantidad total de partidos del club. Si alcanzan el corte, la probabilidad es 100 %; si no, es el porcentaje que representan del corte.

- **S08** - Proyección por xG - Se simulan los partidos que faltan 1.000 veces, usando el xG promedio de cada club para estimar sus goles. La probabilidad es el porcentaje de simulaciones en las que el club termina entre los clasificados.

Diagrama de Casos de Uso

(diagrama de casos de uso disponible en el repositorio)

Casos de usos

CU01	
Nombre	Administrar datos del sistema
Actor	Administrador
Precondición	Para modificar, consultar o dar de baja, el dato tiene que existir.
Flujo Principal	1) El administrador elige que administrar (torneo, club, jugador, árbitro, estadio o entrenador) y la operación (alta, modificación, consulta o baja). 2) Ingresa o modifica datos. 3) El sistema valida que estén completos y sean correctos. 4) El sistema guarda los cambios y confirma.
Flujo alternativo	• 3.a) Hay datos inválidos o incompletos: el sistema muestra error y vuelve al paso 2.

CU02	
Nombre	Gestionar contratos
Actor	Administrador
Precondición	El jugador y club tienen que existir

Flujo Principal	1. El administrador selecciona el jugador. 2. Elige la operación alta, transferencia o rescisión. 3. El sistema actualiza el contrato y guarda el historial.
Flujo alternativo	Si el jugador ya tiene un contrato vigente, no se puede dar de alta otro.

CU03	
Nombre	Inscribir club en torneo
Actor	Administrador
Precondición	El torneo está planificado y el club existe
Flujo Principal	1. El administrador selecciona el torneo y club. 2. El sistema verifica que el club no esté inscripto en ese torneo. 3. El administrador elige los jugadores del plantel. 4. El sistema valida que esté todo correcto y registra la inscripción.
Flujo alternativo	Si el plantel no tiene entre 18 y 30 jugadores, o el club ya estaba inscripto, el sistema no lo permite.

CU04	
Nombre	Generar Fixture
Actor	Administrador
Precondición	El torneo tiene al menos 4 clubes inscriptos
Flujo Principal	1. El administrador selecciona el torneo. 2. El sistema arma los partidos todos contra todos. 3. El sistema muestra el fixture. 4. El administrador confirma y se crean los partidos (sin fecha, estadio ni terna)
Flujo alternativo	La cantidad de clubes es impar: en cada fecha queda un club libre.

CU05	
Nombre	Programar partido

Actor	Administrador
Precondición	El partido existe en el fixture.
Flujo Principal	1. El administrador selecciona el partido. 2. El administrador asigna fecha, estadio y terna arbitral. 3. El sistema valida los datos (CU14) 4. El sistema guarda el partido como PROGRAMADO.
Flujo alternativo	Si el estadio, algún club o algún árbitro no está disponible, el sistema lo rechaza (validaciones del paso 3)

CU06	
Nombre	Cargar nomina
Actor	Operador del partido
Precondición	El partido está programado.
Flujo Principal	1. El operador selecciona el partido y uno de los clubes. 2. Elige los titulares y los suplentes del plantel inscripto. 3. El sistema valida que todos estén en el plantel. 4. El sistema guarda la nómina. 5. Se repite el proceso con el otro club.
Flujo alternativo	Si un jugador está suspendido, no puede incluirse.

CU07	
Nombre	Registrar partido en vivo
Actor	Operador del partido
Precondición	El partido está programado y con las nóminas cargadas.
Flujo Principal	1. El operador inicia el partido. 2. El operador registra los eventos (goles, remates, tarjetas, cambios y faltas). 3. El operador cierra el partido. 4. El sistema actualiza el resultado y las estadísticas.
Flujo alternativo	Si el evento no es válido (ej: es un cambio y el club ya hizo 5), el sistema lo rechaza.

CU08

Nombre	Anular evento
Actor	Operador del partido
Precondición	El partido está en juego
Flujo Principal	1. El operador selecciona el evento cargado por error. 2. Ingresa el motivo. 3. El sistema registra un evento de anulación asociado al original.
Flujo alternativo	Si el evento ya fue anulado, no se puede anular de nuevo.

CU09	
Nombre	Notificar hecho del partido
Actor	Suscriptor y Canales de comunicación
Precondición	Se registró un gol, una tarjeta roja o finalizó el partido
Flujo Principal	1. El sistema arma la notificación con el partido, minuto y hecho. 2. Busco los suscriptores que siguen a alguno de los clubes. 3. Envía el aviso a cada suscriptor por cada canal que eligió. 4. El sistema registra los envíos.
Flujo alternativo	Si un canal falla, el sistema lo registra y sigue con los demás.

CU10	
Nombre	Administrar suscripción
Actor	Suscriptor
Precondición	
Flujo Principal	1. El suscriptor ingresa su nombre, email y teléfono. 2. Elige los clubes que quiere seguir. 3. Elige los canales por los que quiere recibir los avisos. 4. El sistema guarda la suscripción.
Flujo alternativo	Si no elige ningún canal, el sistema le pide al menos uno. Elige un club que ya seguía: el sistema le muestra un mensaje.

CU11	
Nombre	Gestionar patrocinios
Actor	Administrador
Precondición	El club o torneo existe.
Flujo Principal	1. El administrador registra un sponsor y elige el destinatario: club o torneo. 2. Le asigna un patrocinio con montos y fechas. 3. El sistema guarda la información.
Flujo alternativo	Si el club ya tiene un sponsor principal vigente, el sistema lo rechaza.

CU12	
Nombre	Emitir reportes
Actor	Analista
Precondición	El torneo debe existir.
Flujo Principal	1. El analista selecciona el torneo. 2. Elige el reporte: tabla de posiciones, goleadores o valla menos vencida. 3. El sistema lo calcula y lo muestra.
Flujo alternativo	Si no hay partidos finalizados, el sistema informa que no hay datos.

CU13	
Nombre	Calcular proyecciones
Actor	Analista
Precondición	El torneo tiene partidos finalizados
Flujo Principal	1. El analista elige el tipo de proyección. 2. Para el xG, elige un partido y un jugador o club, y el sistema suma el xG de sus remates. 3. Para la probabilidad de clasificación, elige un club y el método (promedio de puntos o simulación con xG), y el sistema calcula el porcentaje.

	4. El sistema muestra el resultado.
Flujo alternativo	Si el club todavía no jugó partidos, no se puede proyectar.

CU14	
Nombre	Verificar disponibilidad
Actor	
Precondición	Se ingresaron la fecha, estadio y la terna del partido
Flujo Principal	1. El sistema verifica que el estadio no tenga otro partido dentro de las 3 horas. 2. Verifica que ningún club juegue otro partido dentro de las 48 horas. 3. Verifica que ningún árbitro tenga otro partido ese día. 4. Verifica que ninguno de los árbitros sea hincha declarado del club local o visitante.
Flujo alternativo	Si algo no está disponible, el sistema informa el motivo y no se programa el partido.

Diagrama de Clases Inicial

Relaciones		
1	Persona — Jugador, Entrenador, Árbitro	HERENCIA
2	Torneo 1—0..* Inscripción	COMPOSICIÓN
3	Torneo — Partido	COMPOSICIÓN
4	Evento — Gol, Remate, Tarjeta, Cambio, Falta, Anulación	HERENCIA
5	Partido — Evento	COMPOSICIÓN

6	Inscripción — Jugador	AGREGACIÓN
7	Club — Entrenador	AGREGACIÓN
8	Jugador — Contrato	COMPOSICIÓN
9	Sponsor — Patrocinio	COMPOSICIÓN
10	Inscripción 0..*—1 Club	ASOCIACIÓN
11	Contrato 0..*—1 Club	ASOCIACIÓN
12	Partido 0..*—2 Club	ASOCIACIÓN
13	Partido 0..*—1 Estadio	ASOCIACIÓN
14	Partido 0..*—3 Árbitro (terna)	ASOCIACIÓN
15	Evento 0..*—1 Jugador	ASOCIACIÓN
16	Cambio 0..*—1 Jugador	ASOCIACIÓN
17	Anulación 0..1—1 Evento	ASOCIACIÓN
18	Patrocinio 0..*—0..1 Club	ASOCIACIÓN
19	Patrocinio 0..*—0..1 Torneo	ASOCIACIÓN
20	Suscriptor 0..*—1..* Club	ASOCIACIÓN

Enumeraciones: EstadoTorneo, EstadoPartido, TipoGol, ZonaRemate, ColorTarjeta, Canal.

Otras clases: Nómina, TernaArbitral, Sanción y EnvioNotificacion.

(diagrama de clases inicial disponible en el repositorio)

Diagrama de Secuencia

CU07 - Registrar Partido en vivo

El operador inicia el partido, registra los eventos mientras está EN JUEGO (con validación y rechazo de eventos inválidos), se notifican los goles y tarjetas rojas a los suscriptores por sus canales, y finalmente cierra el partido.

(diagrama de secuencia 1 disponible en el repositorio)

CU10 - Administrar Suscripción

El suscriptor ingresa sus datos, elige los clubes que sigue (si ya seguía alguno, se le avisa) y los canales (debe elegir al menos uno), y el sistema guarda la suscripción.

(diagrama de secuencia 2 disponible en el repositorio)

Aplicación de principios SOLID

1) **Principio: Responsabilidad Única (S)**
    a) **Problema:** en el diagrama inicial, la clase “Torneo” hace demasiadas cosas: inscribe clubes, genera el fixture, arma la tabla de posiciones, calcula goleadores, valla menos vencida y proyecciones. Posee un montón de funciones que pueden sobrecargar la clase.
    b) **Decisión:** se saca un par de métodos de la clase Torneo. Torneo solo queda con las inscripciones, el estado y el fixture. Los reportes pasan a una nueva clase llamada ServicioReportes.
    c) **Clases:** Torneo y ServicioReportes.
    d) **Ventaja:** las clases son más chicas y fáciles de entender. Un cambio en los reportes no afecta la organización del torneo.

2) **Principio: Cerrado/Abierto (O)**
    a) **Problema:** en el diagrama, hay dos lugares donde agregar algo nuevo obliga a modificar código que ya funciona:
        i) **Canales**: en el diagrama de secuencia del CU07, la clase “Notificacion” envía cada aviso con un switch sobre el enum Canal (APP_MOVIL/CORREO/SMS). Para sumar WhatsApp hay que modificar ese switch, y la RN21 pide lo contrario.

ii) **Proyecciones:** en el diagrama de clases inicial, Torneo tiene el método “calculaProbabilidadClasificacion(club, metodo: String), que elige el cálculo con un “if”. Un método nuevo obliga a modificar Torneo (RN28).

b) **Decisión:**
   i) Crear la interfaz “CanalNotificacion”, implementada por AppMovil,CorreoPrensa y Sms.
   ii) Crear la interfaz “MetodoProyeccion”, implementada por ProyeccionPorPromedio y ProyeccionPorXG.

c) **Clases:** “CanalNotificacion” con sus 3 implementaciones y “MetodoProyeccion” con sus 2 implementaciones.

d) **Ventajas:** un canal o un método nuevo es una clase nueva y no se toca lo que ya funciona.

3) **Principio: Inversión de Dependencias (D)**
   a) **Problema**: en el diseño inicial, como se ve en el CU07, Notificación depende directamente de cada canal concreto, porque tiene que saber cómo se envía por app, por correo y por SMS. Cualquier cambio en un canal obliga a modificar Notificación, y no se la puede probar por separado sin enviar mensajes reales.
   b) **Decisión:** se crea ServicioNotificaciones que trabaja solo con la interfaz CanalNotificacion. De la misma forma, ServicioReportes trabaja con la interfaz MetodoProyeccion.
   c) Clases: ServicioNotificaciones, CanalNotificacion, ServicioReportes, MetodoProyeccion.
   d) **Ventaja:** ServicioNotificaciones y ServicioReportes, que contienen la lógica del negocio, no dependen de los canales ni de los métodos de proyección concretos, sino de sus interfaces. Se puede cambiar o agregar un canal o un método sin modificar esas dos clases, y se las puede probar por separado.

Aplicación de patrones GRASP

1) **Principio: Experto en Información**
   a) **Problema:** en el diagrama de clases, Partido guarda los goles en dos atributos aparte (golesLocal y golesVisitante), aunque esa información ya está en su lista de eventos. El mismo dato está en dos lugares: si se anula un gol y no se actualizan esos atributos, el marcador queda mal.

Ejemplo con el diseño inicial: River mete un gol a los 10’, Boca a los 50’ y luego River mete otro a los 75’ pero se estaba en offside y se lo anulan. Partido guarda el

marcador en dos lugares: en su lista de eventos (3 goles y la anulación) y en los atributos “golesLocal=2” y “golesVisitante = 1”. Al anular el gol del minuto 60, la lista de eventos ya refleja la anulación. Pero si nadie se acuerda de restar 1 a golesLocal, ese atributo sigue diciendo 2. El resultado es: el sistema muestra River 2 – Boca 1, cuando en realidad es 1 a 1. La tabla de posiciones y los goleadores también quedan mal.

b) **Decisión:** cada pregunta la responde la clase que tiene los datos para hacerlo:
   i) Partido calcula los goles de un club con “getGoles(club)”, recorriendo sus eventos, y se eliminan los atributos “golesLocal” y “golesVisitante”.
   ii) Suscriptor responde si sigue a un club con “sigueA(club), porque tiene su lista de clubes, como se ve en el diagrama de secuencia del CU10.
c) **Clases**: Partido, Evento, Gol y Suscriptor
d) **Ventajas:** cada dato vive en un solo lugar, así que no puede quedar desactualizado. Las demás clases no necesitan saber cómo está armada Partido o Suscriptor por dentro: solo le preguntan y reciben la respuesta.

2) **Principio: Creador**
   a) **Problema**: En el diseño inicial, había que definir qué clases serían responsables de crear los objetos del sistema, para evitar que se generen inscripciones, partidos o contratos sin cumplir las reglas de negocio.
   b) **Decisión:** cada objeto lo crea la clase que lo contiene o que tiene los datos para crearlo:
      i) Torneo: crea sus Inscripciones de los clubes y los Partidos del fixture.
      ii) Jugador: crea sus contratos mediante el método registrarContrato(), verificando que no tenga otro contrato vigente incompatible.
   c) **Clases:** Torneo, Inscripción, Jugador y Contrato.
   d) **Ventajas:** De esta manera, las clases responsables de crear los objetos pueden verificar las condiciones necesarias antes de hacerlo. Por ejemplo, torneo comprueba que un club no esté inscripto previamente antes de registrar una nueva inscripción. Esto permite evitar datos inválidos y mantener las reglas de negocio.

3) **Principio: Controlador**
   a) **Problema:** en el diagrama de secuencia CU07, el operador le habla directo a Partido. Eso significa que la interfaz del usuario tendría las clases del modelo (Gol, Partido, Notificacion) y el orden en que se

usan.Si después quieren cambiar el menú por consola por una interfaz gráfica, habría que rehacer toda esa lógica.

b) **Decision:** se agregan Controladores (ControladorTorneo, ControladorPartido y ControladorReportes), que reciben las acciones del usuarios y las delegan en las clases del modelo.
c) **Clases:** ControladorTorneo, ControladorPartido y ControladorReportes.
d) **Ventajas:** la interfaz queda separada del modelo. Se puede reemplazar el menú por consola por una interfaz gráfica sin modificar las clases del negocio.

4) **Principio: Polimorfismo**
   a) **Problema:** cada tipo de evento, canal o método de proyección se comportan de diferentes maneras. Ejemplo; a cualquier evento se le puede pedir “obtenerDescripcion()” y cada tipo responde distinto:
      i) Gol: “Gol de Messi (min 35)”
      ii) Tarjeta: “Tarjeta Roja para Marcos Rojo (min 4)”
      iii) Cambio: “Sale Andrada, entra Pereira (min 60)”.

      Sin polimorfismo, para mostrar un evento habría que preguntar con if o switch si es un gol, una tarjeta o un cambio, y repetir esa pregunta en cada lugar donde se use. Para agregar un tipo nuevo habría que modificar todos esos lugares.
   b) **Decisión:** cada subclase o implementación responde al mismo mensaje a su manera:
      i) Gol, Tarjeta, Cambio, Falta y Anulación sobrescriben obtenerDescripcion(), y Gol además golesPara(club);
      ii) AppMovil, CorreoPrensa y Sms implementan `enviar()`;
      iii) ProyeccionPorPromedio y ProyeccionPorXG implementan calcular().
   c) **Clases**: Evento y sus subclases, CanalNotificacion y sus implementaciones, MetodoProyeccion y sus implementaciones.
   d) **Ventaja:** desaparecen los condicionales por tipo. Por ejemplo, para describir cualquier evento alcanza con evento.obtenerDescripcion(), y un tipo nuevo de evento, canal o método se agrega sin modificar el código que lo usa.

Diagrama de Clases Final

(diagrama de clases final disponible en el repositorio)

Nota de cambios respecto al diagrama de clases inicial

En el diagrama de clases final realizamos varios cambios a partir de la aplicación de los principios SOLID y los patrones GRASP.

Primero, separamos algunas responsabilidades. En el diagrama inicial, `Torneo` tenía métodos para generar reportes y calcular proyecciones. Decidimos trasladarlos a una nueva clase llamada `ServicioReportes`, para que `Torneo` se encargue principalmente de administrar los torneos.

También incorporamos las interfaces `CanalNotificacion` y `MetodoProyeccion`, junto con sus respectivas implementaciones. Esto permite agregar nuevos canales de notificación o métodos de proyección sin tener que modificar las clases que los utilizan. Además, agregamos `ServicioNotificaciones` para administrar los envíos mediante la interfaz correspondiente.

En cuanto a GRASP, eliminamos los atributos `golesLocal` y `golesVisitante` de `Partido` y agregamos el método `getGoles()`, para calcular el marcador a partir de los eventos y evitar guardar información duplicada. También incorporamos `sigueA()` en `Suscriptor` y `registrarContrato()` en `Jugador`, para que estas clases se encarguen de operaciones relacionadas con su propia información.

Por otro lado, agregamos `ControladorTorneo`, `ControladorPartido` y `ControladorReportes`, que reciben las solicitudes del usuario y las delegan a las clases correspondientes. Mantuvimos la herencia de `Evento` y sus subclases, aprovechando el polimorfismo mediante `obtenerDescripcion()`.

Por último, incorporamos el estado `sinProgramar` y ajustamos algunas multiplicidades de `Partido`, ya que al generar el fixture todavía no están asignados todos los datos del partido.

Con estas modificaciones buscamos que las responsabilidades estén mejor distribuidas y que sea más sencillo agregar funcionalidades o realizar cambios en el sistema.